

DoS Stress-Testing & Auto-Mitigation for AI Inference Servers

Cost-aware admission control in front of a vLLM endpoint

Team Phishers

Mena Abdelmoneim · Salama Albedwawi · Rewan Ramadan

vLLM 0.29.0 / Qwen2.5-0.5B-Instruct / NVIDIA Tesla T4
github.com/mena-04/DoS-Stress-Testing

Against a Locust flood of 28 attacker identities on a real T4, the queue cap did exactly its job — peak backend queue **24 → 0** — and the gateway still refused **92%** of legitimate requests, while its p95 *improved* to 0.63 s. One mis-set cost threshold did that. We know because the harness measures legitimate success, not latency.

24 → 0

peak backend queue depth, mitigation on

8.3%

of legitimate requests succeeded

0.63 s

the p95 that failure reported

49 : 322

token-cost gap one threshold missed

The unit of load on an inference server is the **token**, not the request. Every rate limiter in common use disagrees.

`nginx limit_req`, API-gateway requests-per-second rules and WAF throttles all count **requests**, because on a conventional web service requests cost roughly the same. On an LLM endpoint they do not: a 24-token chat reply and a 2048-token generation differ in GPU cost by nearly three orders of magnitude, and both are one well-formed request.

Who deploys this

A team self-hosting an open-weights model behind a public or partner-facing API, without per-tenant GPU isolation — the normal case, since dedicating a GPU per tenant is what self-hosting avoids.

What it replaces

It takes the position an API gateway already occupies, speaks the same OpenAI-compatible protocol, and swaps the requests-per-second rule living there for a **token-cost budget**. The endpoint URL moves by one port; the model, the server and the client are untouched.

The attack needs no volume

decode, measured on T4	0.035 s / token
one 1024-token generation	≈ 36 s
vLLM launched with	<code>--max-num-seqs 8</code>
8 requests hold the GPU for	36 s

Eight requests is a rate no plausible RPS budget would block and no anomaly rule would call a flood. No botnet, no spoofing, no volume — just a large `max_tokens`.

WHY THIS APPROACH NOW / 01

vLLM's continuous batching and paged KV cache make the scarce resource **countable** — sequence slots and KV blocks, with a known limit — rather than diffuse CPU time. There is a number to admit against.

WHY THIS APPROACH NOW / 02

vLLM exports `num_requests_running` and `num_requests_waiting` on a Prometheus endpoint, so a proxy can read the backend's **real queue** instead of inferring it from response times, which is a lagging signal.

WHY NOT A LEARNED DETECTOR ALONE

Shedding has to be correct on the **first** request of an attack it has never seen, and a model cannot promise that. The two required mechanisms are deterministic and auditable; the VAE only tunes their parameters.

02

Proposed solution

two required mechanisms + one optional



MECHANISM 01 · 429

Dual token bucket, charged on cost

One bucket meters request rate. The second meters **cost**, charged prompt_tokens + max_tokens **at admission** — the true cost is only known once a generation ends, by which time the GPU time is spent. An omitted max_tokens is charged the default, not zero; n and best_of multiply the charge.

MECHANISM 02 · 503

Queue cap with fast rejection

global_max_inflight is held **at** the backend's --max-num-seqs. Admit more and the queue still forms, but inside vLLM where it cannot be reordered or shed. Past the cap a request waits 200 ms, then gets a 503. A reserved lane partitioned on **request cost**, not identity, is what survives ID rotation — but only while its threshold sits between the two cost distributions. Slide 5 is what happens when it does not.

MECHANISM 03 · OPTIONAL

VAE novelty detection

A variational autoencoder over a 12-feature per-client window, trained **on normal traffic only** and scoring reconstruction error as novelty — so no attack is ever labelled and nothing leaks across the train/test boundary. It tunes bucket multipliers rather than blocking, decays instead of latching, and fails open.

Every refusal is legible, and immediate

A caller that waits 30 seconds for a 503 has already been denied service, so each rejection carries its reason and a Retry-After.

429	rate_limit_requests · rate_limit_cost · client_concurrency · cost_exceeds_budget · anomaly_shed
503	queue_pressure · queue_timeout · global_capacity
413	body_too_large

Three rules that protect the measurement, not the server

1. The gateway never reads X-Traffic-Label. Branching on ground truth would turn "attacker rejection rate" into a measurement of the label rather than of the defence.
2. Mitigation-off still goes through the gateway, so the proxy hop appears in both arms. A baseline that bypassed it would credit the mitigation with removing a cost it never added.
3. The generator never retries a 429 or 503. A retrying client turns rejection into amplification and makes the rejection rate meaningless.

03

Solution validation

the harness is the deliverable that worked

TARGET SERVICE

Real vLLM 0.29.0, Qwen2.5-0.5B, Tesla T4, `--max-num-seqs 8`. Real weights, real Prometheus metrics.

LOAD

Locust, four profiles. Headline pair is `flood`: 4 legitimate users against 28 attackers for 30 s, seed 42.

PAIRED ARMS

Same generator, same GPU, one config flag. `off` traverses the gateway too, so the proxy hop is in both.

EVIDENCE

Every number recomputed from raw per-request logs, not console output. 211 state samples, **0% stale**.

STATISTICAL CARE

Percentiles over successful requests only, and always beside the success rate. A fast 503 is a failure, not a quick success.

AUTOMATED TESTS

74 CPU tests against a stand-in calibrated to the measured T4 timing. Passing them is not GPU evidence.

The mitigation was the easy half. Every finding below came from a test that looked like it was passing.

- a
- The shipped thresholds failed the flood. The queue cap held perfectly — peak backend queue 0 — while 88 of 96 legitimate requests got 503 `queue_timeout`. Cause on slide 5: legitimate cost 48–49, attacker 320–322, `cheap_cost_threshold` 512, so both counted as cheap and the reserved lane partitioned nothing. Open: set the threshold from the measured distribution, then rerun.
- b
- Queue depth read zero when the queue was deepest. The sampler scraped `/metrics` inside a bare `except: pass` and read a missing value as 0 — and `/metrics` is what stops answering under load. Fixed: absent is unknown, never empty. The flood pair above records 0% staleness, which is how we can trust its queue of 0.
- c
- Identity rotation defeats every per-client defence. Confirmed again at 28 attacker identities: each one sits inside its own budget, so the buckets never fire and the global cap degenerates to first-come-first-served — which the attacker wins, arriving 20× more often. Design answer: partition on cost, not identity. See (a) for the condition that makes it work.
- d
- Fair-share inverts under long requests. Cost aged out of the 10 s window while the request ran 36 s, so an attacker holding every slot was charged 0.0 and a steady cheap client measured as 100%. Diagnosed and patched, **not merged**: this snapshot has 74 tests and does not contain the fix.

REPRODUCE — NO GPU

`pip install -e "[dev,report]"`
`pytest` → 74 passed
Or just open `results/`: the CSVs and charts are committed.

REPRODUCE — ON THE GPU

Run `final_demo.ipynb` with `RUN_LIVE_DEMO = True`: it serves real vLLM, runs the paired OFF/ON flood under new run IDs, and packages the raw logs into `results/demo_evidence.zip`.

ONE RUN KEPT OUT OF THE RESULTS

A pass where every request returned `ConnectError` because no gateway was listening. Its 35,128 attacker "requests" are refused connections returning instantly, not load. It sits in `excluded_runs` with the reason rather than deleted.

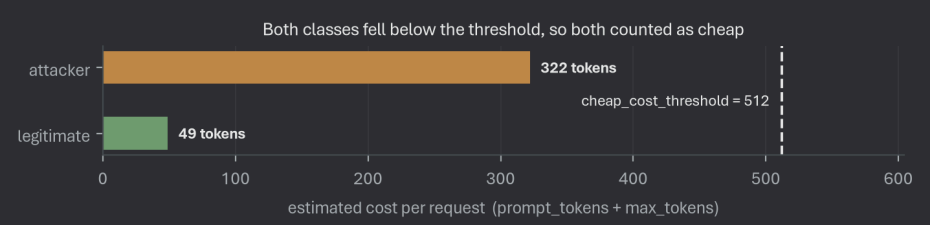
04

Results and conclusions

flood profile · same load, same GPU, one config flag apart

LEGITIMATE USERS, ATTACK WINDOW	OFF	ON
Success rate	100%	8.3%
Attempted / succeeded	33 / 33	96 / 8
Rejected, all queue_timeout	0	88
p95 over successes	2.986 s	0.630 s
Peak backend queue	24	0
Attacker rejection rate	0%	88.9%

Both required mechanisms fired. The queue cap emptied the backend queue, and 88.9% of attacker traffic was refused. It was still not a success: the cost-partitioned lane is what should have kept legitimate traffic flowing, and one number stopped it.



Rate-limit the right quantity

Cost, not requests. All 28 attackers stayed inside a per-client RPS budget while holding the GPU.

Hold the queue where you can act on it

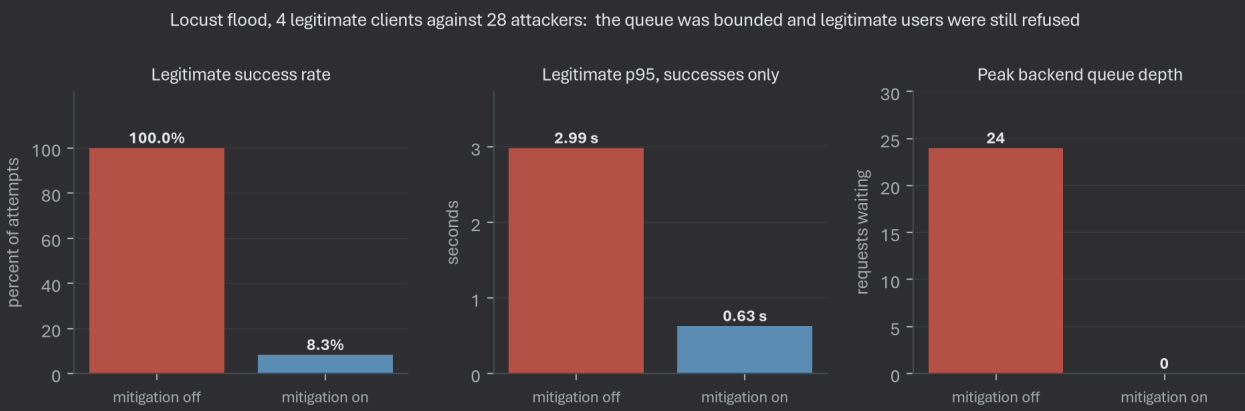
Capping admission at `--max-num-seqs` took the queue from 24 to 0. This part worked, first time.

A cost partition needs a measured threshold

512 was right for one attack shape and inverted for the next. It belongs between the two observed cost distributions.

Measure availability, not latency

Our own p95 said we had won. Only success rate caught it — the harness is what we would reuse.



The middle and right panels are what a dashboard would show, and both say it worked: latency down 4.7×, queue gone. The left panel is the one that matters. A gateway that refuses 92% of legitimate traffic has an excellent p95 *because* it refused them — which is why every percentile here is printed beside its success rate, and why we report this run and not the small single-attacker probe that flattered us.